National University of Computer and Emerging Sciences

AL-2002: Artificial Intelligence Lab
Lab Manual 11

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives

After performing this lab, students shall be able to:

- ✔ Understand the concepts of overfitting and underfitting
- ✔ Interpret learning curves
- ✔ Apply k-fold cross-validation
- ✔ Understand and implement early stopping
- ✔ Understand dropout and its role in regularization
- ✔ Improve model performance and generalization

1 Introduction

In machine learning, building a model is not just about achieving high accuracy on training data. A good model should generalize well to unseen data.

Two common problems arise during training:

- Overfitting: Model performs well on training data but poorly on new data
- Underfitting: Model fails to capture patterns even in training data

To address these issues, several techniques are used, including:

- Learning curves
- Cross-validation
- Regularization (dropout)
- Early stopping
- Hyperparameter tuning

1.1 Underfitting

Underfitting occurs when the model is too simple to learn the underlying patterns.

Characteristics:

- Low training accuracy
- Low validation accuracy
- Model cannot capture relationships

Example Causes:

- Model too simple
- Insufficient training
- Poor feature selection

1.2 Overfitting

Overfitting occurs when the model learns noise instead of patterns.

Characteristics:

- Very high training accuracy
- Low validation accuracy
- Poor generalization

Example Causes:

- Model too complex
- Too many parameters
- Small dataset

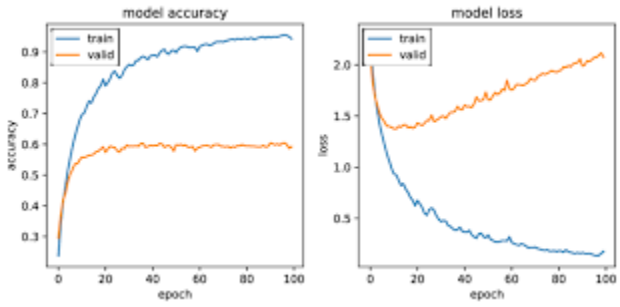
2 Learning Curves

Learning curves help visualize model performance over time.

They plot:

- Training loss (or accuracy)
- Validation loss (or accuracy)

For example:

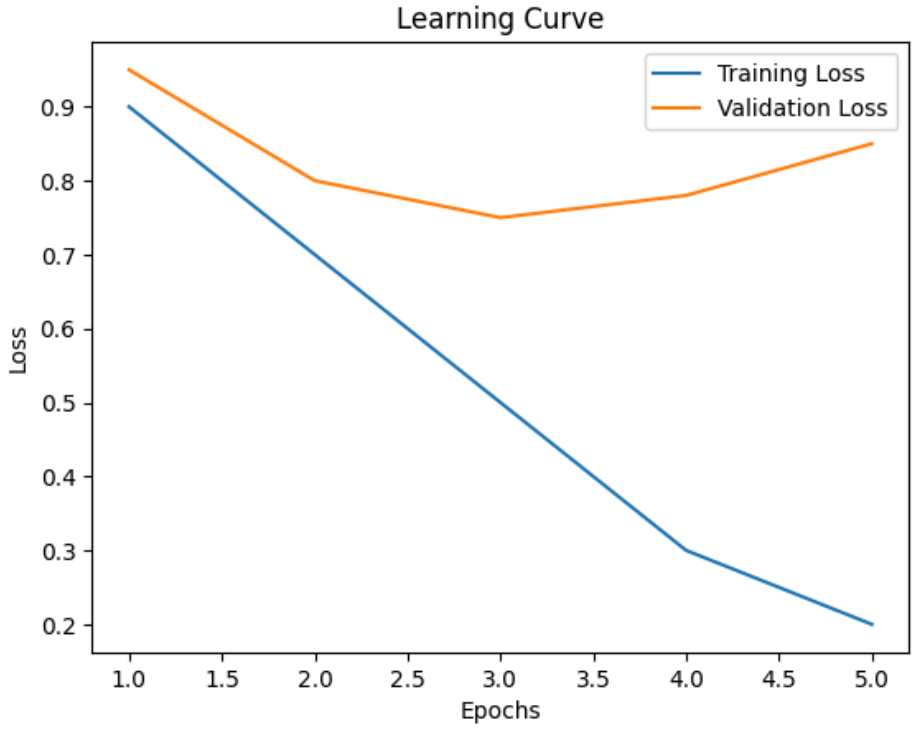


Scenario	Training Error	Validation Error	Meaning
Underfitting	High	High	Model too simple
Good Fit	Low	Low	Ideal
Overfitting	Low	High	Model too complex

Simple learning curves can be plotted as follows:

```
import matplotlib.pyplot as plt
epochs = [1,2,3,4,5]
train_loss = [0.9, 0.7, 0.5, 0.3, 0.2]
```

```
val_loss = [0.95, 0.8, 0.75, 0.78, 0.85]

plt.plot(epochs, train_loss, label="Training Loss")
plt.plot(epochs, val_loss, label="Validation Loss")

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Learning Curve")
plt.legend()
plt.show()
```



While using tensorflow, you can simply extract this loss. Consider the example given below:

Model Training

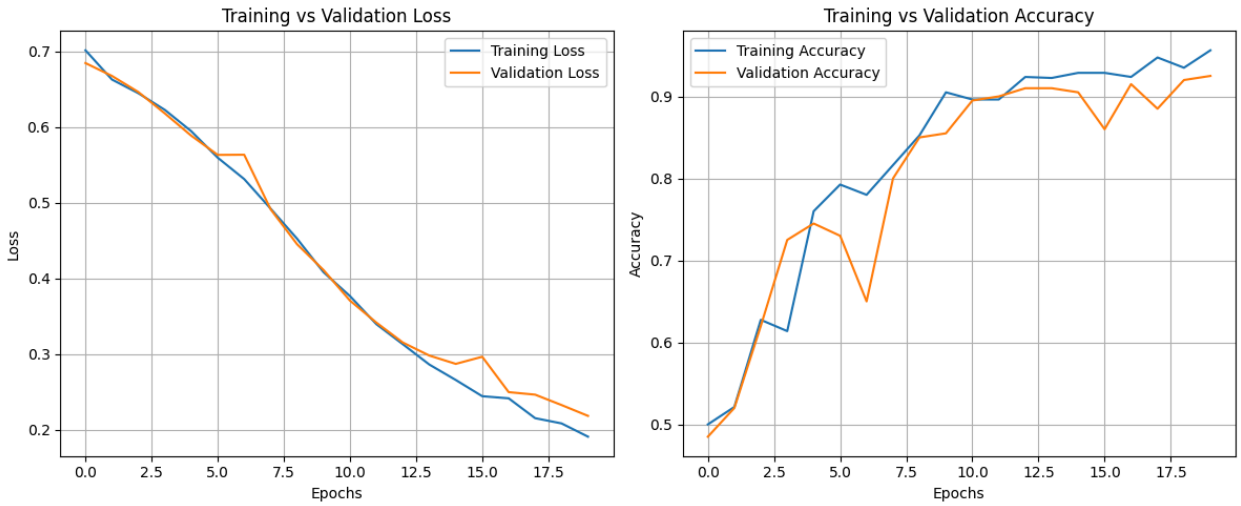
```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------
```

```
# 1. Generate Sample Dataset
# ------------------------------
# Create random input features
x = np.random.rand(1000, 10)

# Create binary labels based on a simple rule
y = (np.sum(x, axis=1) > 5).astype(int)

# ------------------------------
# 2. Build Neural Network Model
# ------------------------------
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(10,)),  #
hidden layer
    tf.keras.layers.Dense(32, activation='relu'),                     #
hidden layer
    tf.keras.layers.Dense(1, activation='sigmoid')                    #
output layer
])

# ------------------------------
# 3. Compile Model
# ------------------------------
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',   # loss for binary classification
    metrics=['accuracy']          # track accuracy
)

# ------------------------------
# 4. Train Model
# ------------------------------
history = model.fit(
    x, y,
    epochs=20,
    batch_size=32,
    validation_split=0.2,    # 20% data used for validation
)
```

Learning Curves

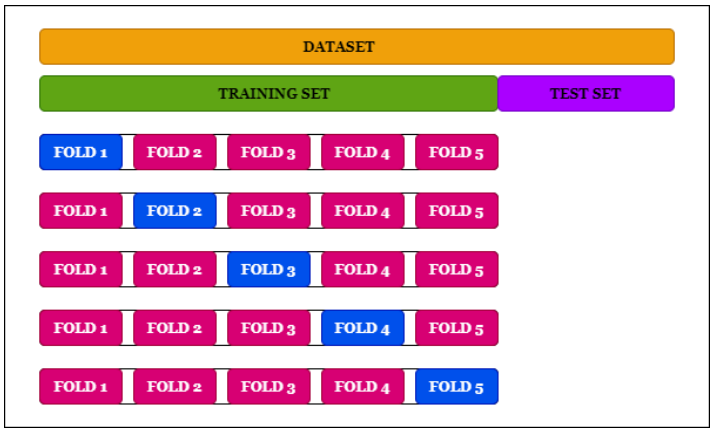
```
# ------------------------------
# 5. Extract Learning Curves
# ------------------------------
train_loss = history.history['loss']          # training loss
val_loss = history.history['val_loss']        # validation loss
train_acc = history.history['accuracy']       # training accuracy
val_acc = history.history['val_accuracy']     # validation accuracy

# ------------------------------
# 6. Plot Loss Curve
# ------------------------------
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# ------------------------------
# 7. Plot Accuracy Curve
# ------------------------------
plt.subplot(1, 2, 2)
plt.plot(train_acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# ------------------------------
# 8. Display Plots
# ------------------------------
```

```
plt.tight_layout()
plt.show()
```



3 K-Fold Cross Validation

K-Fold Cross Validation is a statistical technique to measure the performance of a machine learning model by dividing the dataset into K subsets of equal size (folds). The model is trained on K − 1 folds and tested on the last fold. This process is repeated K times, with each fold being used as the testing set exactly once. The performance of the model is then averaged over all K iterations to provide a robust estimate of its generalization ability.



Cross-validation serves multiple purposes:

- Avoids Overfitting: Ensures that the model does not perform well only on the training data but generalizes to unseen data.

- Provides Robust Evaluation: Averages results over multiple iterations, reducing bias and variance in the performance metrics.
- Efficient Use of Data: Maximizes the utilization of the dataset, especially when the data size is limited.

A dataset can be divided into k folds using the `KFold` function from `sklearn.model_selection`

Using Scikit-learn

```
from sklearn.model_selection import KFold
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

data = load_iris()
X, y = data.data, data.target

kf = KFold(n_splits=5)

model = LogisticRegression(max_iter=200)

scores = []

for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    scores.append(accuracy_score(y_test, predictions))

print("Cross-validation scores:", scores)
print("Average score:", sum(scores)/len(scores))
```

Using Tensorflow

```
import numpy as np
import tensorflow as tf
from sklearn.model_selection import KFold
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

data = load_iris()
X, y = data.data, data.target

scaler = StandardScaler()
X = scaler.fit_transform(X)

kf = KFold(n_splits=5, shuffle=True, random_state=42)

scores = []
for train_index, test_index in kf.split(X):

    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(16, activation='relu',
input_shape=(X.shape[1],)),
        tf.keras.layers.Dense(8, activation='relu'),
        tf.keras.layers.Dense(3, activation='softmax')  # 3 classes in
Iris
    ])
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    model.fit(X_train, y_train, epochs=30, batch_size=16, verbose=0)
    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

    scores.append(accuracy)

print("Cross-validation scores:", scores)
print("Average score:", np.mean(scores))
```

4 Early Stopping

Early stopping is a regularization technique that stops model training when overfitting signs appear. It prevents the model from performing well on the training set but underperforming on unseen data i.e validation set. Training stops when performance improves on the training set but degrades on the validation set, promoting better generalization while saving time and resources.

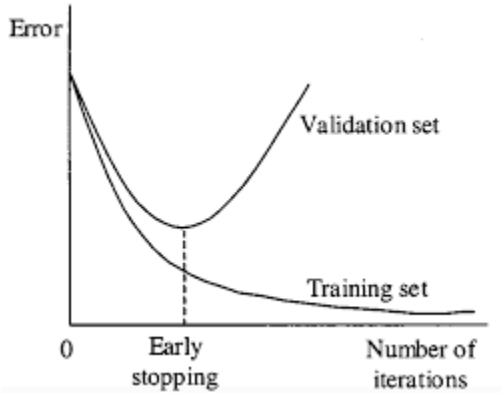
The technique monitors the model's performance on both the training and validation sets. If the validation performance worsens, training stops and the model retains the best weights from the period of optimal validation performance.

Early stopping is an efficient method when training data is limited as it typically requires fewer epochs than other techniques. However, overusing early stopping can lead to overfitting the validation set itself, similar to overfitting the training set.

The number of training epochs is a hyperparameter that can be optimized for better performance through hyperparameter tuning.

Key Parameters in Early Stopping

- Patience: The number of epochs to wait for validation improvement before stopping, typically between 5 to 10 epochs.
- Monitor Metric: The metric to track during training, often validation loss or validation accuracy.
- Restore Best Weights: After stopping, the model reverts to the weights from the epoch with the best validation performance.



It can be added in tensorflow models using the following code:

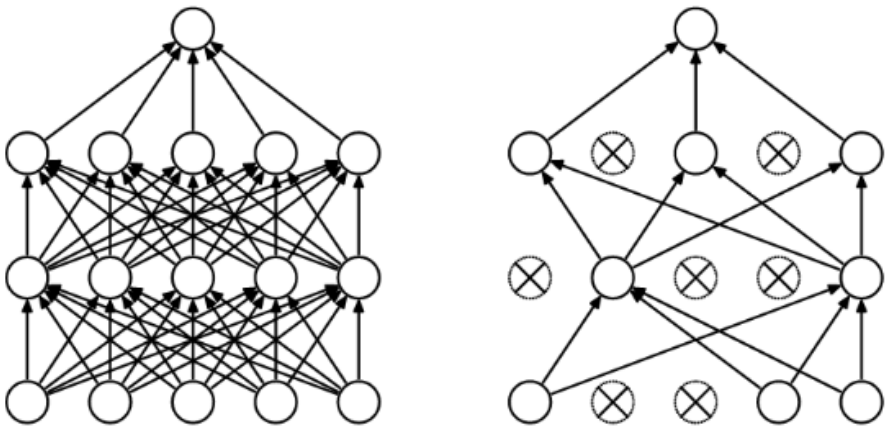
```
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',      # watch validation loss
    patience=3,              # wait 3 epochs before stopping
    restore_best_weights=True
)
```

Once this callback has been defined, it can be simply added to the fit function.

```
history = model.fit(
    x, y,
    epochs=50,               # large number (training may stop early)
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop], # apply early stopping
)
```

5 Dropout

The term "dropout" refers to dropping out the nodes (input and hidden layer) in a neural network. All the forward and backwards connections with a dropped node are temporarily removed, thus creating a new network architecture out of the parent network. The nodes are dropped by a dropout probability of p.



(a) Standard Neural Net (b) After applying dropout.

In the overfitting problem, the model learns statistical noise. To be precise, the main motive of training is to decrease the loss function, given all the units (neurons). So in overfitting, a unit

may change in a way that fixes up the mistakes of the other units. This leads to complex co-adaptations, which in turn leads to the overfitting problem because this complex co-adaptation fails to generalise on the unseen dataset.

Now, if we use dropout, it prevents these units to fix up the mistake of other units, thus preventing co-adaptation, as in every iteration the presence of a unit is highly unreliable. So by randomly dropping a few units (nodes), it forces the layers to take more or less responsibility for the input by taking a probabilistic approach.

This ensures that the model is getting generalised and hence reducing the overfitting problem.

Dropout can be applied to a network using TensorFlow APIs as follows:

```
tf.keras.layers.Dropout(
    rate
)
# rate: Float between 0 and 1.
# The fraction of the input units to drop.
```

Consider the example below:

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(10,)),
    tf.keras.layers.Dropout(0.5),   # Drop 50% of neurons randomly
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.3),   # Drop 30% of neurons
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

It can be incorporated by simply adding another layer in your ANN

6 When to use what

We explored different techniques to improve model performance and generalization. Each method plays a specific role depending on the problem observed during training.

Technique	When to Use	Problem Addressed	Key Idea
Learning Curves	During/after training to analyze performance	Diagnosis tool	Compare training vs validation trends
K-Fold Cross-Validation	When dataset is small or evaluation is unreliable	Poor generalization estimate	Train/test multiple times on different splits
Early Stopping	When validation performance starts degrading	Overfitting during training	Stop training at optimal point
Dropout	When model overfits (especially deep networks)	Overfitting	Randomly disable neurons to improve generalization

A good machine learning model is not the one that performs best on training data, but the one that generalizes well to unseen data. In practice, these techniques are often used **together** such as learning curves for diagnosis, cross-validation for evaluation and regularization methods like dropout and early stopping for improving performance.